

COMS20018, Introduction to AI: Generative Models

Nan Lu

School of Computer Science
University of Bristol

6th October 2026

Agenda

- ▶ What is generation? Discriminative vs. generative models
- ▶ Three ways to generate
 - ▶ Variational autoencoders (VAE)
 - ▶ Generative adversarial networks (GAN)
 - ▶ Diffusion models
- ▶ Autoregressive models

Today's question

How can a machine learn from examples and then generate something it has never seen before?

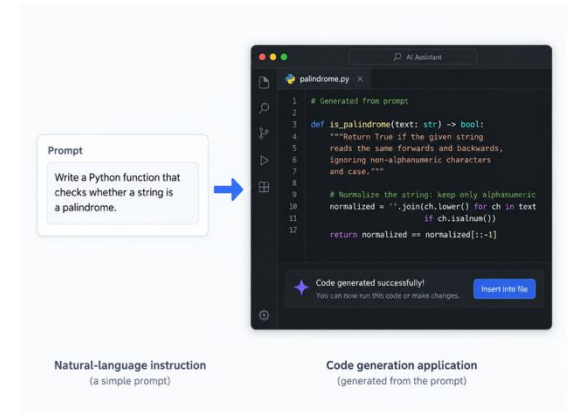
What can generative AI generate?



A photograph of a person who does not exist



A song generated specifically for this lecture



A working application generated from a natural-language description

- ▶ None of these outputs existed before someone asked for it.

Where did these outputs actually come from?

Three possible explanations

A. Retrieval

The system found an existing example and returned it.

Then every output would be an exact copy of something. It is not.

B. Hand-coded rules

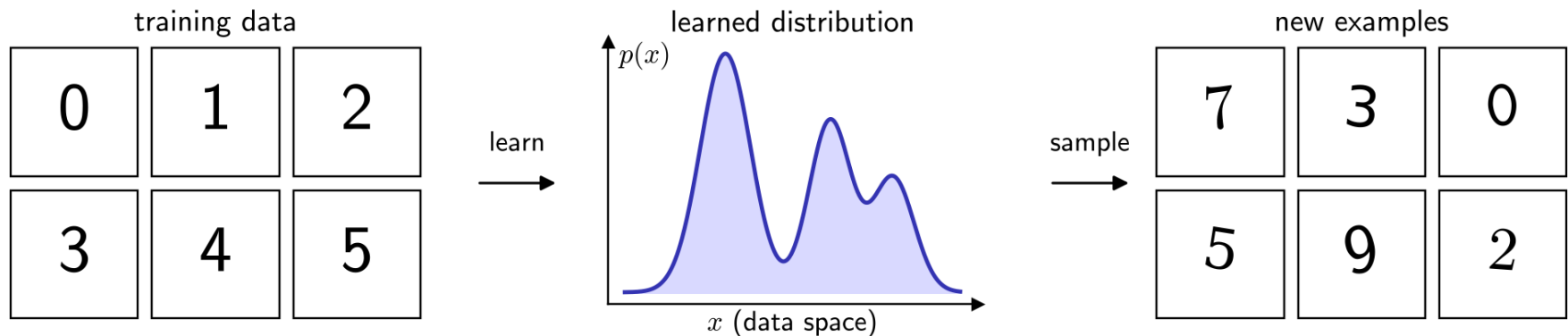
A programmer wrote rules describing how to construct the output.

Then someone wrote rules for every possible request. Nobody did.

C. Learned generation

The system learned statistical patterns from many examples and used them to produce a new sample.

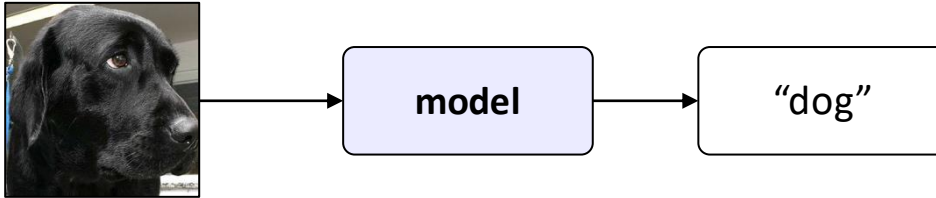
This is the idea behind generative models.



⇒ Learn the distribution of the training data, then *sample* from it: the new examples follow the same patterns but are not copies.

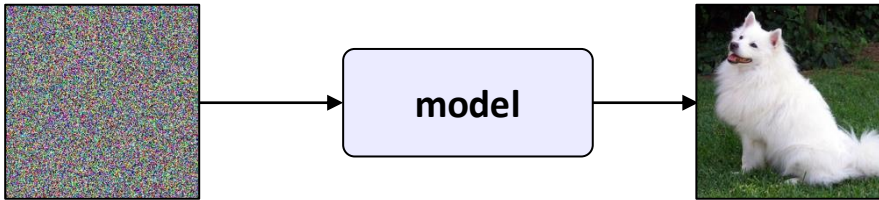
Two different jobs

Discriminative



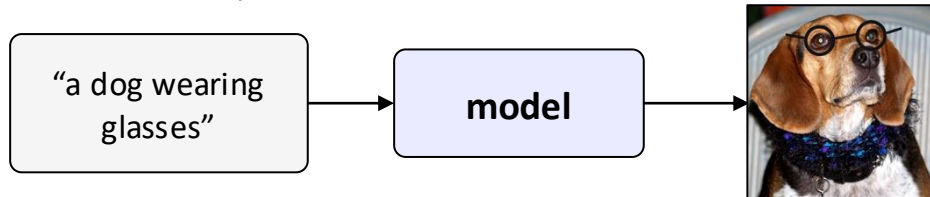
decide something about an example

Generative



produce a new example that could have come from the data

Generative, conditional



*generate, given what we ask for: the **condition** decides what kind of dog*

- ▶ Discriminative: example in, decision out. This is what you have done so far (L02–L05).
 - ▶ Generative: the model produces the data itself.
 - ▶ Conditional generation is what a text prompt does: the prompt is the condition.
- ⇒ Producing an example is much more demanding than labelling one.

What each kind of model learns

Discriminative model

learns $P(y | x)$

- ▶ the probability of a label y given an observed input x
- ▶ use: predict a label for x
- ▶ e.g. logistic regression, a neural network classifier

Generative model

learns $P(x)$ or $P(x | y)$

- ▶ how the data x are distributed: unconditionally, or given a label y
- ▶ use: generate new examples x
- ▶ e.g. VAEs, GANs, diffusion models (we will see soon)

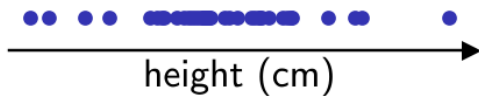
- ▶ $P(x)$: generate a plausible example from the data distribution.
 - ▶ $P(x | y = \text{"dog"})$: generate a plausible example of a dog.
 - ▶ Watch the bar: a discriminative model conditions *on* x ; a generative model describes x itself (possibly given y).
- ⇒ A discriminative model learns only $P(y | x)$: it says nothing about how x is distributed, so it cannot generate new data.

The split is a useful teaching distinction rather than a strict taxonomy.

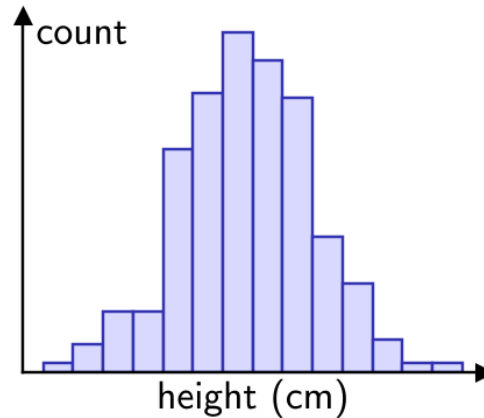
What does it mean to “learn a distribution”?

Start with one number per example: the heights of a large number of people.

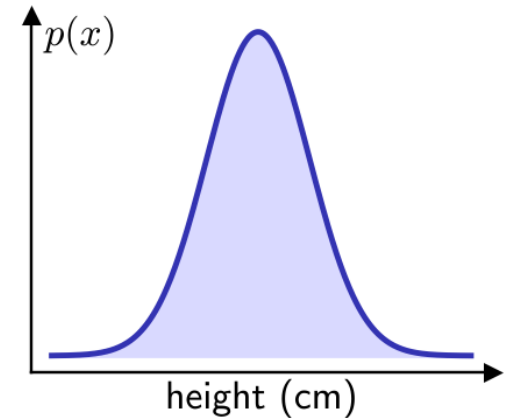
1. Observe some data



2. Count how often values occur

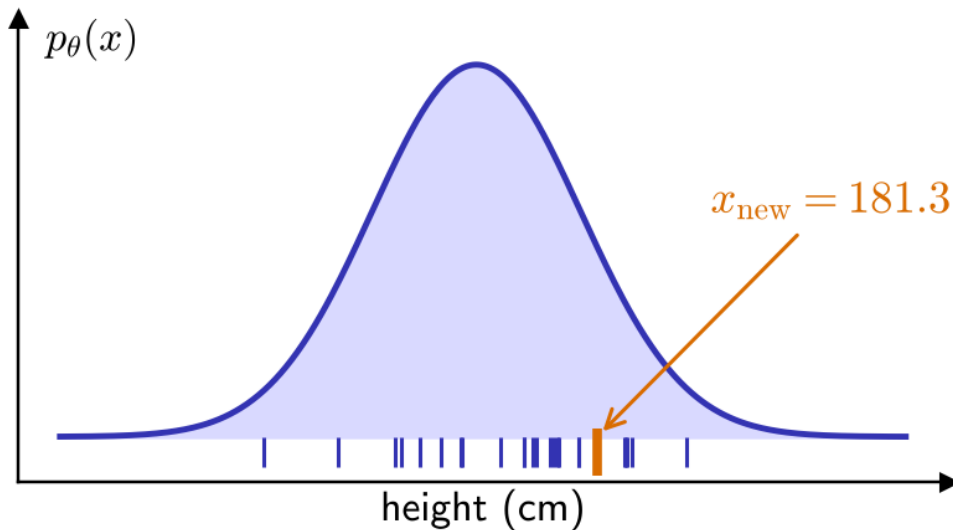


3. Fit a distribution



- ▶ The fitted curve gives every height a plausibility, including heights we never observed.
 - ▶ “Learn a distribution” means learning which values are plausible and how likely they are.
 - ▶ The model does not store individual data points; it summarises how the data are distributed.
- ⇒ Generative modelling applies the same idea to much more complex data: images, audio, video and text.

Generation is sampling



Sampling

$$x_{\text{new}} \sim p_{\theta}(x)$$

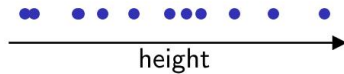
draw a new example from the distribution the model learned

θ : the parameters learned from the training data

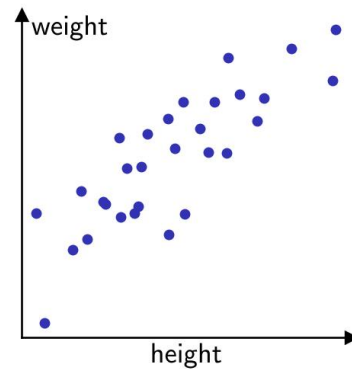
- ▶ High-probability regions are sampled often; low-probability regions rarely.
 - ▶ Sample again and you may get a different value: the model is unchanged, only the random draw differs.
 - ▶ A sample does not need to appear in the training data.
- ⇒ Generative modelling = learning a distribution + sampling from it

From one number to high-dimensional data

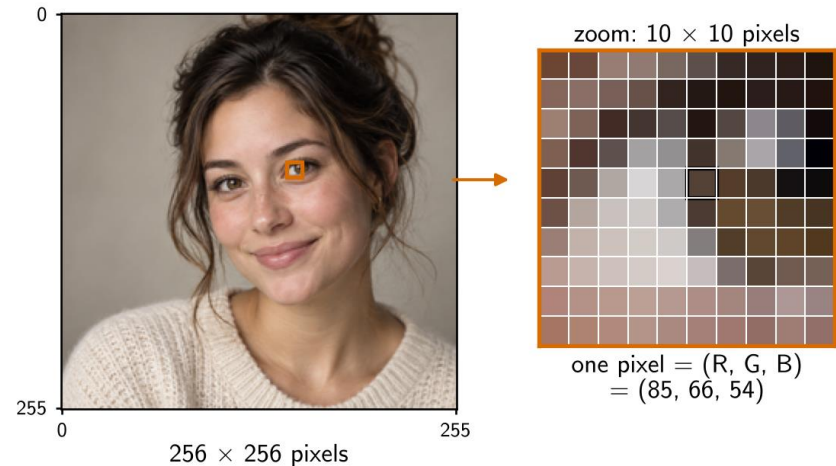
One variable



Two variables



Many variables: a colour image



$x = (x_1, x_2, \dots, x_n)$ a 256 × 256 colour image has $256 \times 256 \times 3 = 196,608$ values

- ▶ The challenge is not only the number of variables: they are highly structured and depend on one another (neighbouring pixels are similar).
 - ▶ Choose 196,608 pixel values independently at random and you get noise, almost never a meaningful image.
- ⇒ A generative model has to learn the structure of valid images.

The curse of dimensionality

What changes when the number of variables becomes very large?

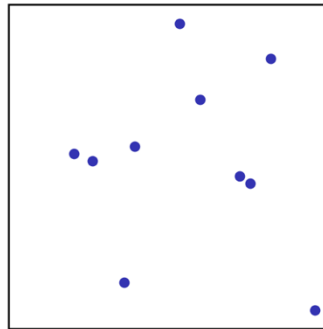
1 dimension



dense

the same 10 observations
cover the line well

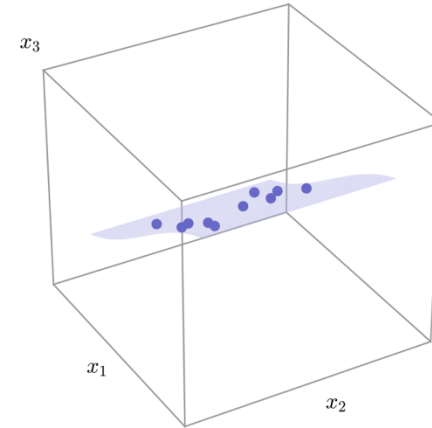
2 dimensions



sparse

the same 10 observations, in
a much larger space

196,608 dimensions



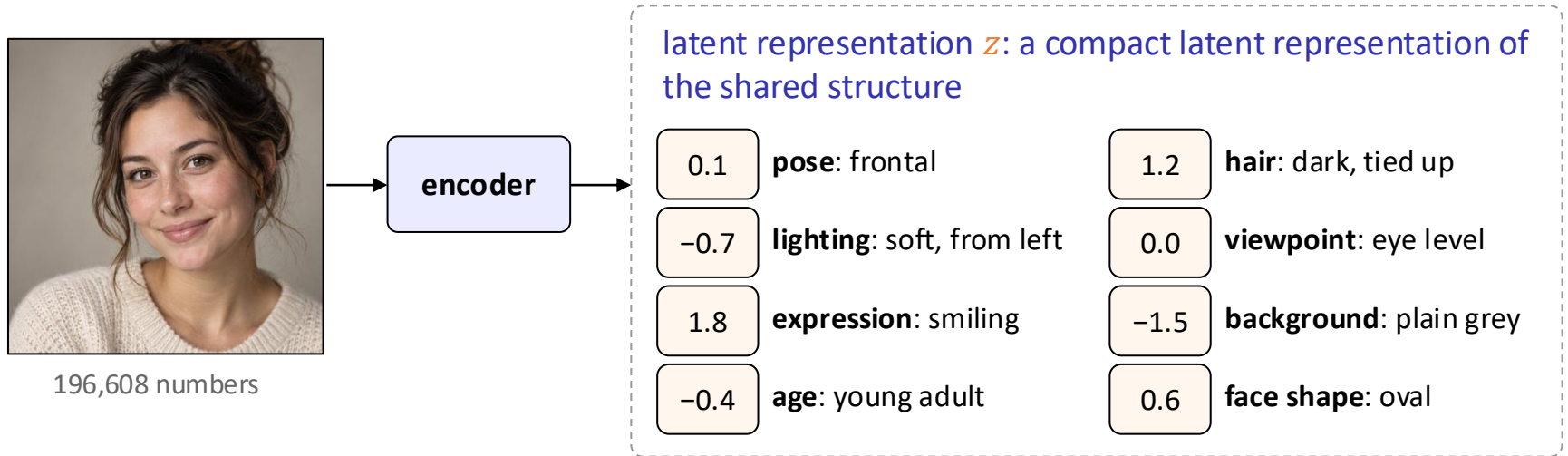
extremely sparse

3 of the 196,608 axes: 10
points, almost all of the
space empty

- ▶ As dimensionality grows, the space expands far faster than the data we can collect, so training examples become extremely sparse.
 - ▶ But real data occupy only a small, structured region of that space.
- ⇒ Can we learn that structure with a simpler representation?

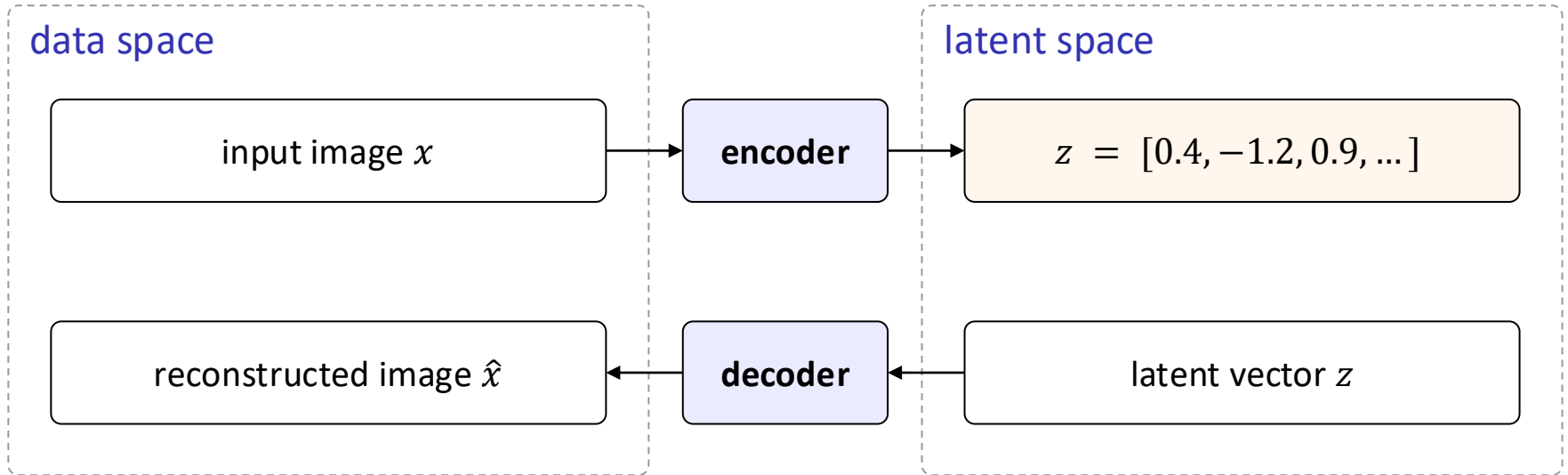
Faces are not random pixels: latent representations

The face from slide 3 has 196,608 pixel values, but it varies along far fewer underlying factors:



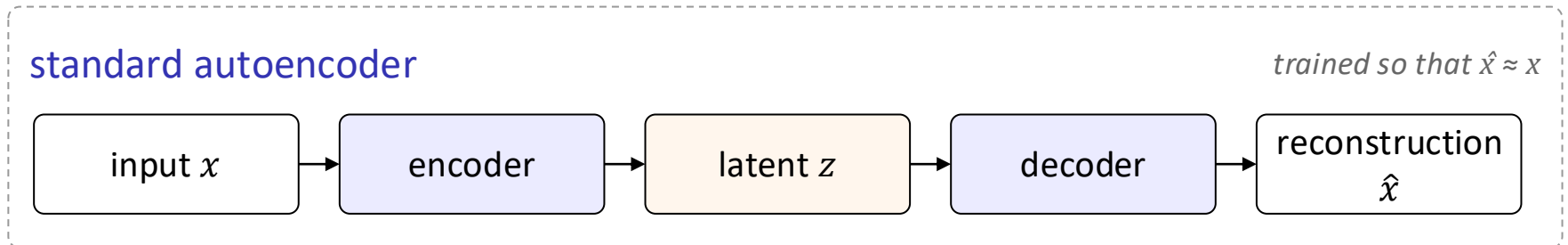
- ▶ **Latent** means hidden: not directly observed in the data, but inferred by the model.
 - ▶ z is a short vector capturing the important underlying variation, instead of representing every pixel directly.
 - ▶ The model learns z from examples; the factors above illustrate the idea and are not labels we provide.
- ⇒ Describing a face to a sketch artist takes a few dozen words, not 200,000 numbers.

Encoder and decoder



- ▶ Encoder: compress x into a latent representation z .
 - ▶ Decoder: map a latent representation z back to data space.
 - ▶ Train both together so that the reconstruction $\hat{x}$ resembles the input x : an **autoencoder**.
- ⇒ If we can choose meaningful latent vectors z , the decoder can generate new examples.
- ? But how do we make sure a randomly chosen z decodes to something meaningful?

From autoencoder to variational autoencoder (VAE)



The problem

- ▶ Each input is mapped to a single latent point.
- ▶ The latent space can be irregular, with gaps between useful regions.
- ▶ A randomly chosen z often lands in a gap, and the decoder outputs nonsense.

What a VAE adds

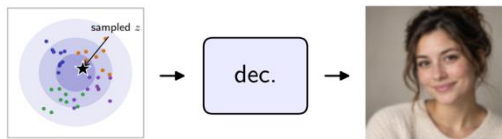
- ▶ The encoder outputs a *distribution* over z , not a single point.
- ▶ Sample z from that distribution during training.
- ▶ Pull the latent space towards a simple prior, usually $N(0, I)$.

Three approaches, one goal

The shared goal

Learn the structure of the training data well to generate new, plausible samples.

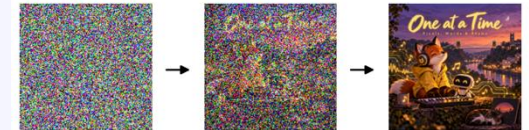
Variational autoencoder (VAE) Kingma & Welling, 2013



Generative adversarial network (GAN) Goodfellow et al., 2014

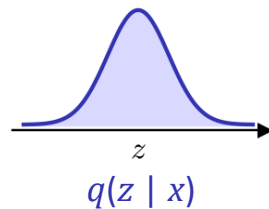
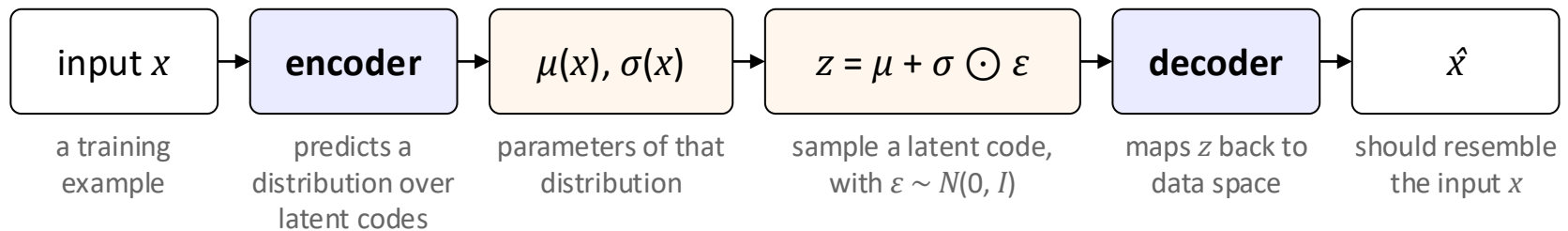


Diffusion model Sohl-Dickstein et al., 2015; modern DDPM 2020



- ▶ Same goal, different mechanism: latent-variable learning, adversarial competition, or iterative denoising.
- ▶ These models are core building blocks behind many modern generative AI tools.
- ▶ For each one, ask: *what does it start from, and what does it learn?*

How does a variational autoencoder work?



A plain autoencoder outputs *one* latent point. A VAE outputs a *distribution* over latent points, $q(z | x)$.

1. Reconstruct the input

$$\hat{x} \approx x$$

z must carry enough information to rebuild the training example.

2. Organise the latent space

$$q(z | x) \approx N(0, I)$$

The latent distributions stay close to a simple prior.

- ⇒ A VAE learns representations that reconstruct the data while keeping the latent space smooth and easy to sample from.
- “Variational”: $q(z | x)$ is a learned approximation to the difficult posterior $p(z | x)$.

The Evidence Lower Bound (ELBO)

How do we balance good reconstruction with a well-structured latent space?

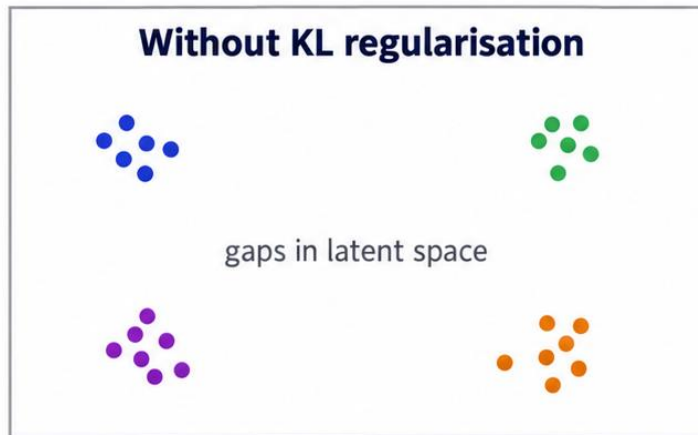
$$L_{\text{ELBO}} = \mathbb{E}_{q(z|x)} [\log p(x|z)] - D_{\text{KL}}(q(z|x) \parallel p(z))$$

Reconstruction term

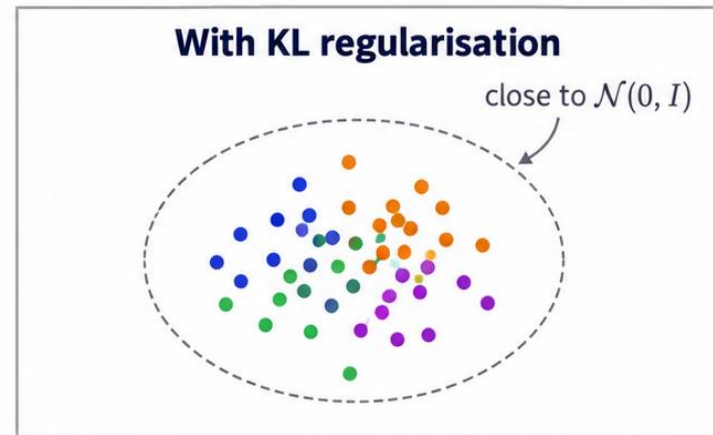
*Can the decoder rebuild x from z ?
Keep useful information about
the example*

latent-space regularization term

*KL divergence: a distance-like measure
between two probability distributions
Keep $q(z|x)$ close to $\mathcal{N}(0, I)$*



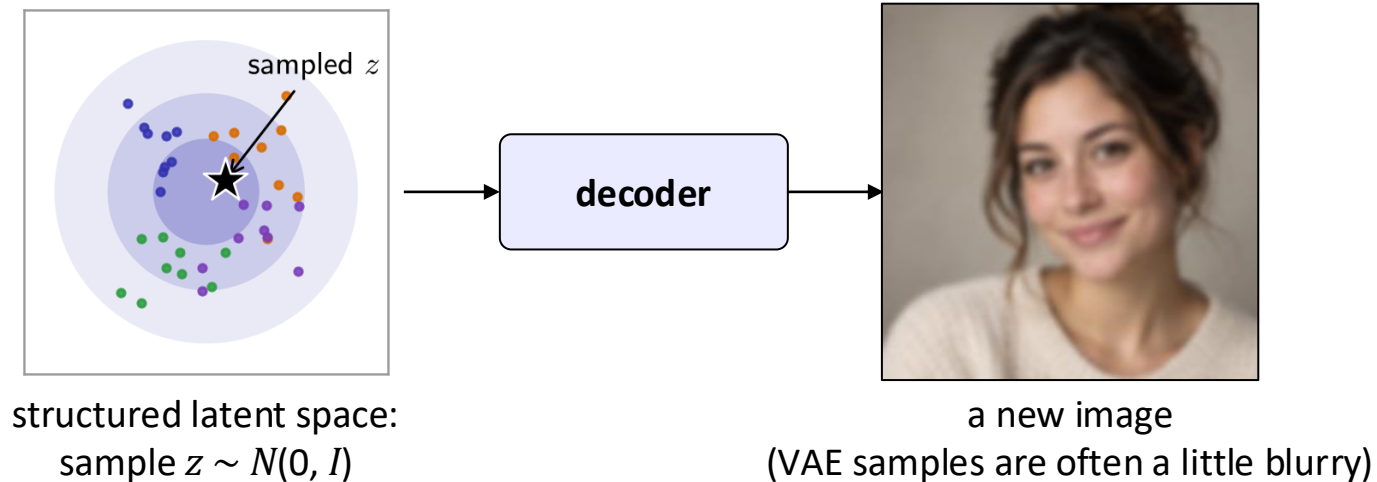
random z may land in empty regions
and decode badly



sampling becomes smoother
and more reliable

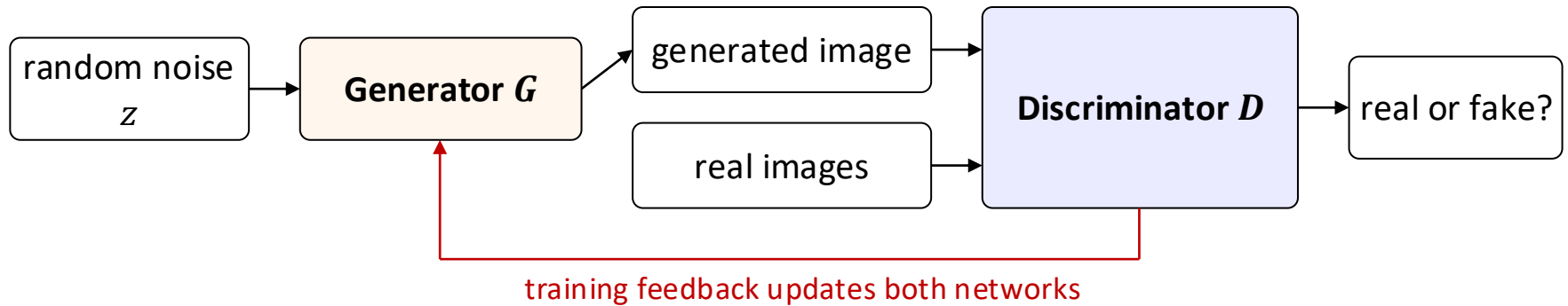
⇒ Maximise ELBO = reconstruct the data well + keep the latent space easy to sample from

Generating with a VAE



- ▶ The encoder was only scaffolding: it helped build the latent space and is not needed to generate.
 - ▶ Sample a different z and you get a different image, from the same fixed model.
 - ▶ VAE samples tend to be coherent but somewhat blurry: trade a little reconstruction accuracy for a smooth, sampleable latent space.
- ⇒ **Central intuition:** learn a structured latent space, then sample from it and decode.

GANs: learning through competition



The generator asks

"Can I make samples convincing enough to fool the discriminator?"

The discriminator asks

"Can I still tell the real examples from the generated ones?"

- ▶ Adversarial: the two networks have opposing objectives.
 - ▶ No hand-written rule defines a good image: D learns from real examples, and G learns from D 's feedback.
- ⇒ **Central intuition:** improve a generator by competing with a critic.

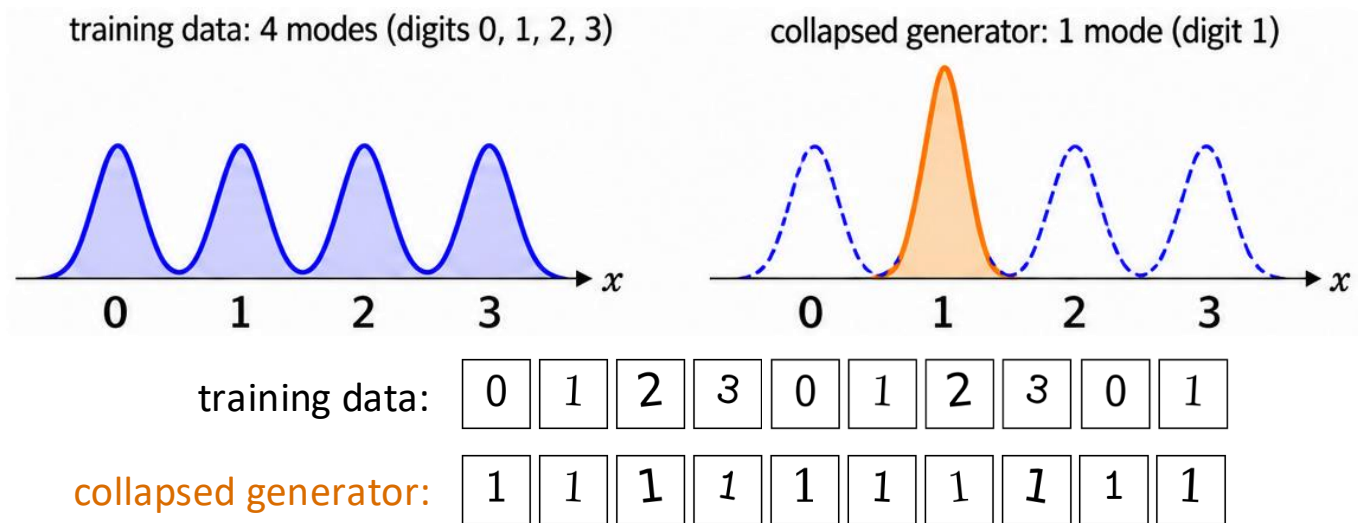
When competition goes wrong: mode collapse

Hard to train

- ▶ Two networks improve against each other.
- ▶ If one gets too far ahead, the other stops learning.

Mode collapse

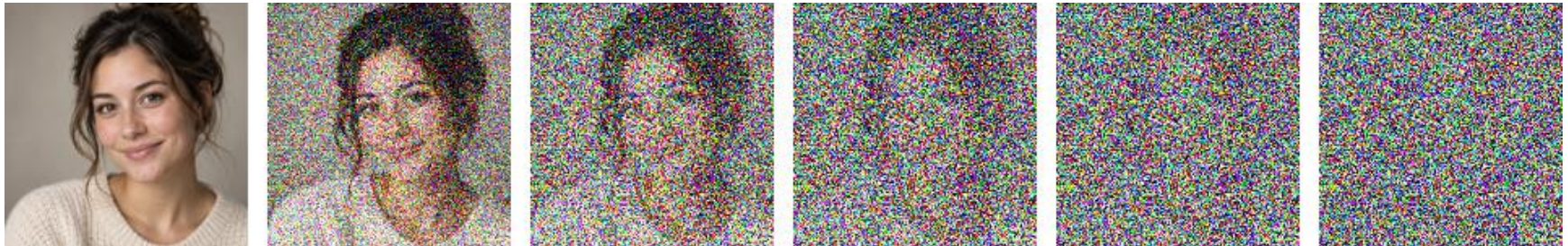
The generator produces realistic samples, but only from a **small subset of the modes** of the training data: it repeats a few kinds of example and has very little variety.



- ▶ A *mode* is a peak of the data distribution: one kind of example (e.g., one digit).
 - ▶ Cause: G finds a few outputs that reliably fool D and keeps producing them.
- ⇒ Convincing individual samples $\neq$ representing the whole distribution; quality is not diversity.

Diffusion (1/2): the forward process adds noise

Take a training image and add a little noise. Then a little more. Keep going.



x_0 : real image

x_1

x_2

x_3

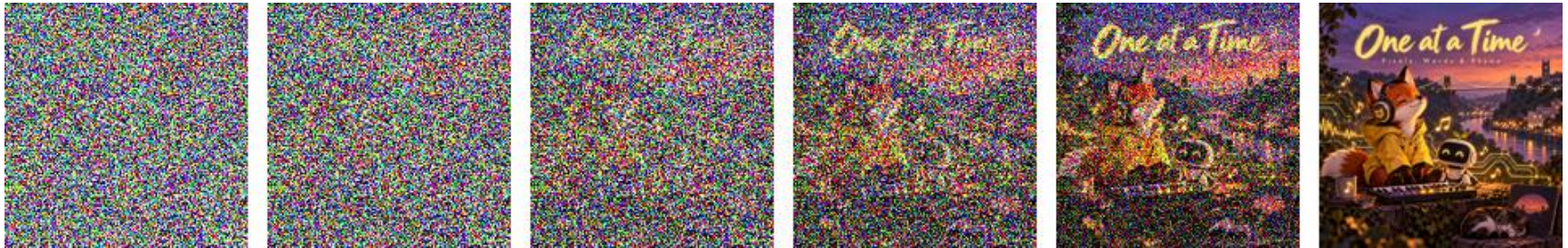
x_4

x_T : pure noise

forward process: fixed, not learned

- ▶ Nothing is learned in this direction: we simply add Gaussian noise, step by step.
 - ▶ It gives an unlimited supply of training pairs: a noisier image and the cleaner one it came from. The data supply their own answer key.
 - ▶ The forward process is only used to create training data.
- ⇒ Can a network learn to undo one small step of noise?

Diffusion (2/2): generate by denoising random noise



$x_T \sim N(0, I)$

denoise

denoise

denoise

denoise

x_0 : new image

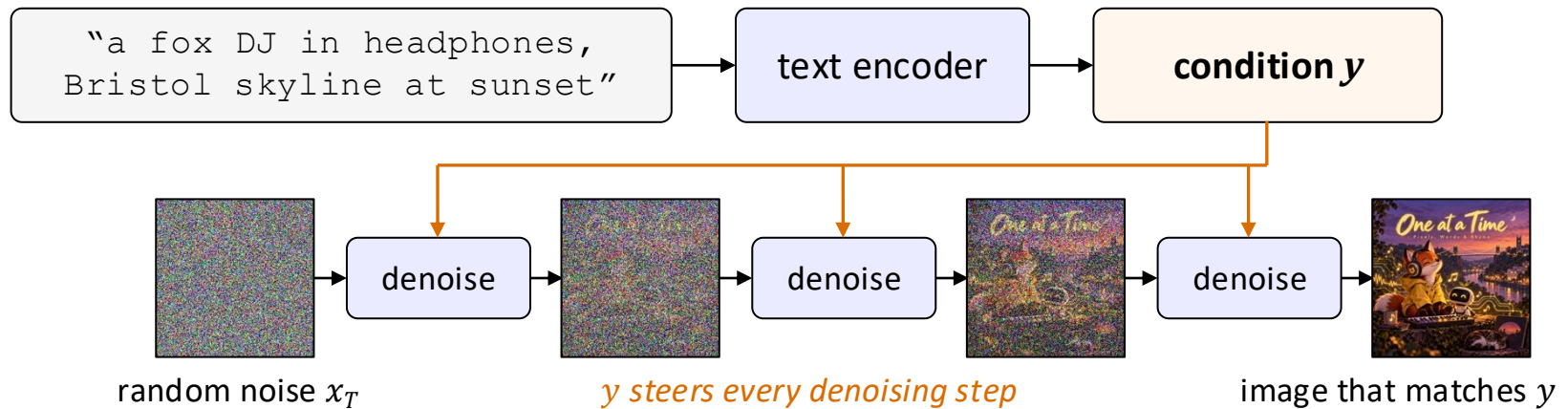
reverse process: the same learned denoising network, applied over and over

Training and generation

- 1. Training:** show the network noisy images from the forward process and teach it to predict the noise to remove.
- 2. Generation:** start from **random noise** $x_T \sim N(0, I)$, not from a training image.
- 3. Repeatedly denoise:** $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$, removing a little noise each step.
- 4. Result:** the final x_0 is a new sample.

⇒ **Central intuition:** learn to remove noise, then use that skill to generate from pure noise.

Conditioning: telling it what to generate

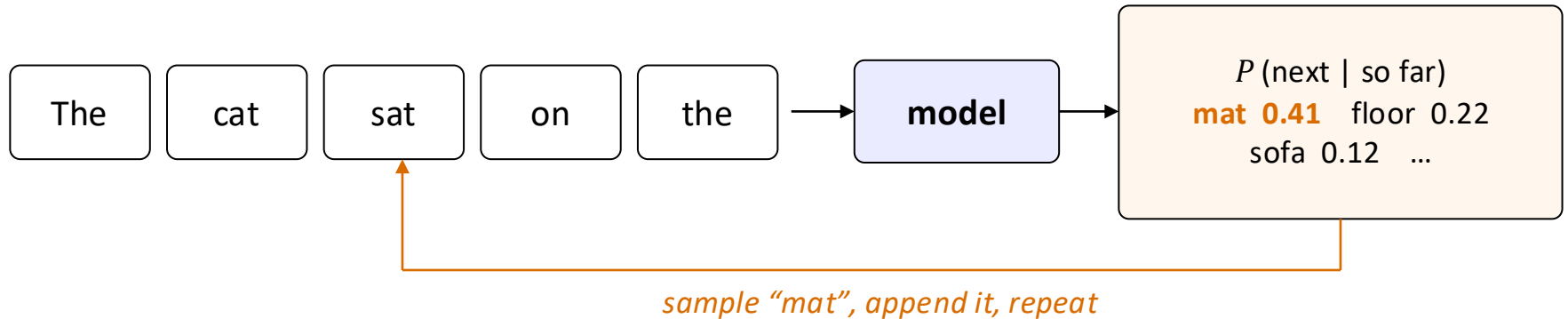


- ▶ Unconditional generation: produce something from the learned distribution, sample from $p(x)$.
- ▶ Conditional generation: produce something given what we asked for, sample from $p(x | y)$.
 - ▶ y can be a class label, a caption, a sketch, another image ...
- ▶ This is the $P(x | y)$ from the start of the lecture, in action.

Autoregressive models: one step at a time

Generate one element at a time, each conditioned on everything generated so far:

$$p(x) = p(x_1) p(x_2 | x_1) p(x_3 | x_1, x_2) \cdots = \prod_i p(x_i | x_{<i})$$



- ▶ The elements can be words (tokens), pixels, or audio samples.
 - ▶ Each step is a prediction followed by a random draw: generation is still sampling.
 - ▶ Training is self-supervised: the data supply the next element as the answer.
- ⇒ With words as the elements, this is how large language models generate text: the topic of next week.

Different mechanisms, same goal

Model	Starts from	Core intuition
VAE	a latent vector $z \sim N(0, I)$	sample from a structured latent space and decode it
GAN	random noise z	a generator improves by competing with a discriminator
Diffusion	random noise x_T	turn noise into data by removing it in small steps
Autoregressive	the elements so far	repeatedly predict and sample the next element

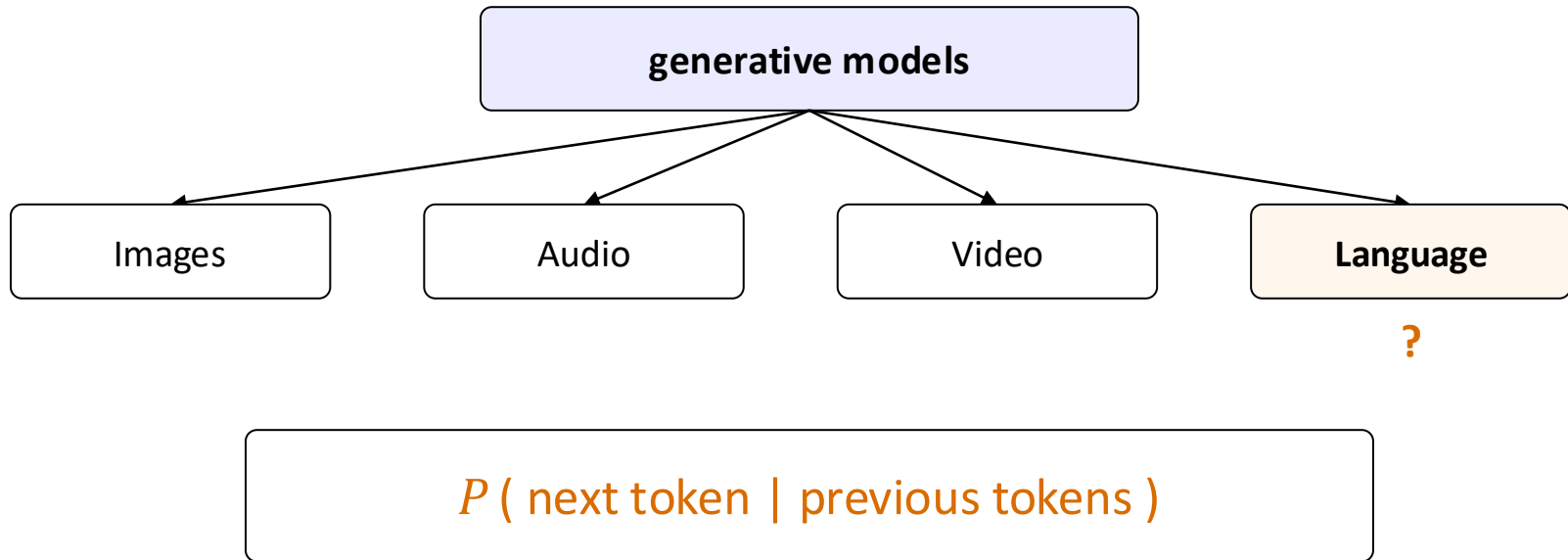
- ▶ No row is the winner: they trade off sample quality, diversity, speed and ease of training.
- ▶ Current practice: diffusion dominates image generation; autoregressive models dominate text. VAEs and GANs remain useful, often inside larger systems.

Summary

Five things to remember

1. Discriminative models learn $P(y | x)$; generative models learn $P(x)$ or $P(x | y)$.
2. Generation = learn a distribution, then sample from it.
3. Latent representations compress high-dimensional data into a few key factors.
4. VAE: decode a sampled latent.
GAN: generator vs. discriminator.
Diffusion models: denoise random noise.
Autoregressive: predict the next element.
5. Plausible is not the same as true or diverse (e.g. GAN mode collapse).

Next week



How can predicting the next token possibly produce something like ChatGPT?

Reading

- ▶ Stanford CS236, *Deep Generative Models* course notes: autoregressive models, VAEs and GANs. The best single companion to this lecture. [Free online.](#)
- ▶ Google for Developers, *Introduction to GANs*: generator and discriminator training, and common problems such as mode collapse. [Free online.](#)
- ▶ Weng, L., *What are diffusion models?* (2021): forward noising and reverse denoising; the introductory sections are enough. [Free online.](#)
- ▶ Kingma, D. P. & Welling, M., *An Introduction to Variational Autoencoders* (2019): latent-variable models and the ELBO (optional). [arXiv:1906.02691.](#)
- ▶ Kingma, D. P. & Welling, M., *Auto-Encoding Variational Bayes* (2013): the original VAE paper (optional, advanced). [arXiv:1312.6114.](#)
- ▶ Ho, J., Jain, A. & Abbeel, P., *Denoising Diffusion Probabilistic Models* (2020): the paper behind the diffusion slides (optional). [arXiv:2006.11239.](#)